Assistentes de AI generativa e o risco de Débito Cognitivo - Exercício

INF1028 e INF1045

Alessandro Garcia
Vicente Neto

Aviso Importante

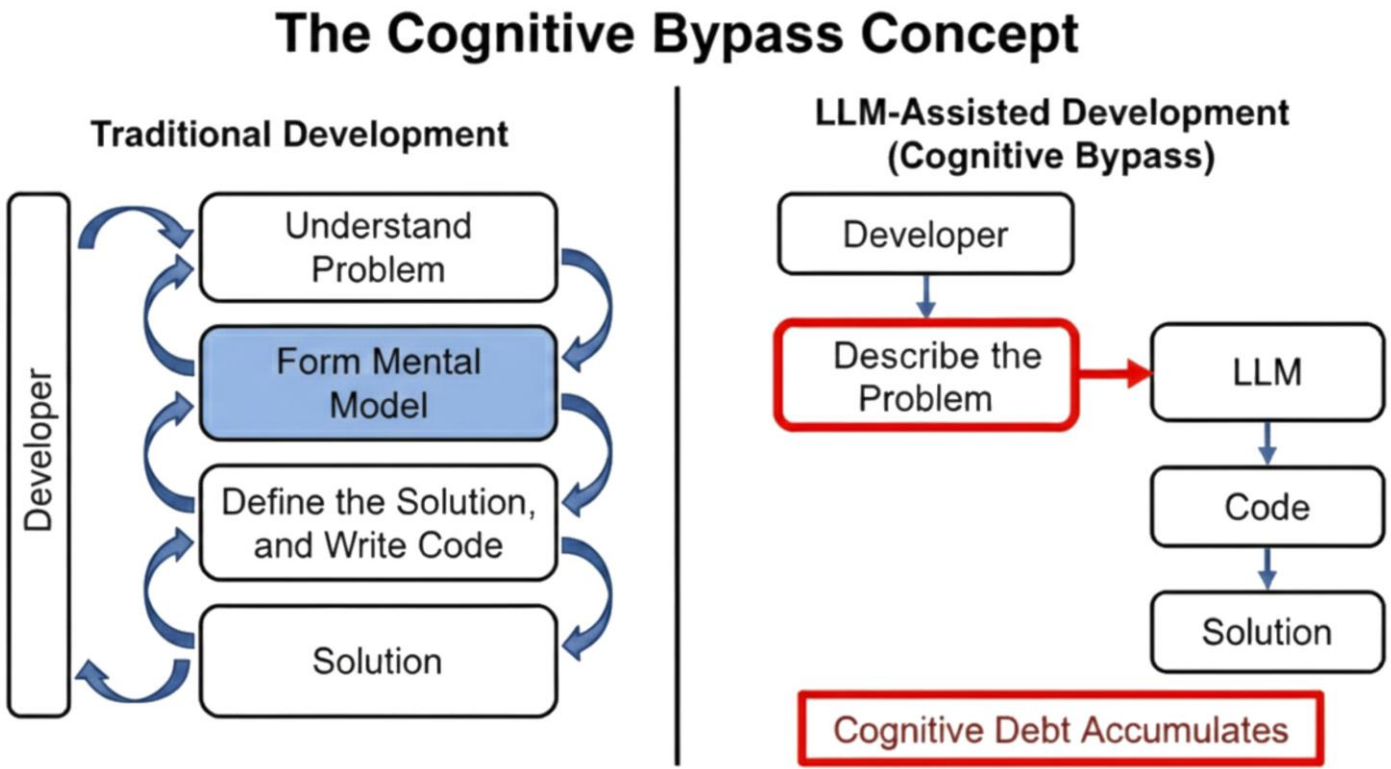
Informar os links para as issues em que estão trabalhando:

•**Até Quinta-Feira**

•**No canal #relatórios**: Cada aluno deve enviar a url para a issue em que está trabalhando**.**

•**Caso mudem de issue:** Informar no mesmo canal #relatórios sobre a mudança e a url.

Motivação - Mitigar o acúmulo de débito cognitivo



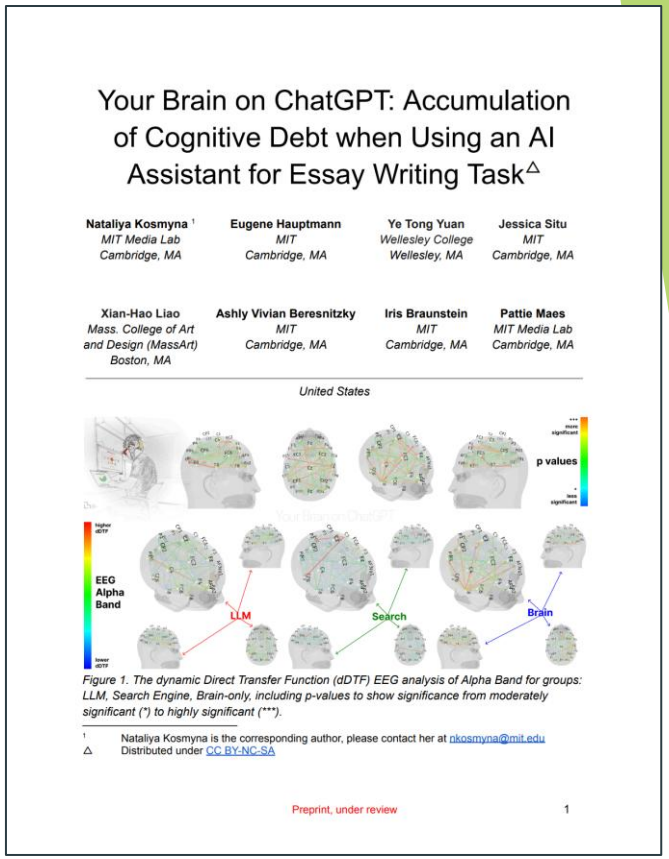
Referências

Abstract

With today's wide adoption of LLM products like ChatGPT from OpenAI, humans and businesses engage and use LLMs on a daily basis. Like any other tool, it carries its own set of advantages and limitations. This study focuses on finding out the cognitive cost of using an LLM in the educational context of writing an essay.

We assigned participants to three groups: LLM group, Search Engine group, Brain-only group, where each participant used a designated tool (or no tool in the latter) to write an essay. We conducted 3 sessions with the same group assignment for each participant. In the 4th session we asked LLM group participants to use no tools (we refer to them as LLM-to-Brain), and the Brain-only group participants were asked to use LLM (Brain-to-LLM). We recruited a total of 54 participants for Sessions 1, 2, 3, and 18 participants among them completed session 4.

We used electroencephalography (EEG) to record participants' brain activity in order to assess their cognitive engagement and cognitive load, and to gain a deeper understanding of neural activations during the essay writing task. We performed NLP analysis, and we interviewed each participant after each session. We performed scoring with the help from the human teachers and an AI judge (a specially built AI agent).



Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task△

Nataliya Kosmyna [1] *MIT Media Lab Cambridge, MA*
Eugene Hauptmann *MIT Cambridge, MA*
Ye Tong Yuan *Wellesley College Wellesley, MA*
Jessica Situ *MIT Cambridge, MA*
Xian-Hao Liao *Mass. College of Art and Design (MassArt) Boston, MA*
Ashly Vivian Beresnitzky *MIT Cambridge, MA*
Iris Braunstein *MIT Cambridge, MA*
Pattie Maes *MIT Media Lab Cambridge, MA*

United States

Figure 1. The dynamic Direct Transfer Function (dDTF) EEG analysis of Alpha Band for groups: LLM, Search Engine, Brain-only, including p-values to show significance from moderately significant (*) to highly significant (***).

[1] Nataliya Kosmyna is the corresponding author, please contact her at nkosmyna@mit.edu
△ Distributed under CC BY-NC-SA

https://arxiv.org/abs/2506.08872

Referências

Abstract

The advent of Large Language Models (LLMs) and the tools they enable have dramatically reduced the cost of code production, effectively allowing developers to create and increment software artifacts with unprecedented ease. However, this convenience introduces undesirable side effects, notably the accumulation of *Cognitive Debt*. This debt arises from the removal of friction—a mechanism that, while uncomfortable, is the very fabric of knowledge construction and retention. Developers can now merge functional code without forming a rigorous *Mental Model* of its logic, leading to systems that are fast to grow but increasingly difficult to maintain. To the best of our knowledge, no existing workflow addresses this challenge head-on.

This paper challenges the prevailing trend of "frictionless" development, arguing that developer expertise thrives not on ease, but on the active exercise of solving hard problems. We propose **Mastery-Aware Pipeline**, a workflow that transforms AI-assisted software production into a disciplined cognitive challenge, re-establishing the developer as a software designer rather than a passive reviewer.

The workflow comprises a pipeline supported by AI Agents with two critical stages: (1) **Formal Specification**, achieved through a `TODO.md` artifact that humans—and humans only—must author to guide development; and (2) **Proof-Of-Mastery**, which requires the developer to process the AI-generated code and `REPORT.md` to produce a `REACTO.md` artifact. This structure, inspired by the homonymous technique used in technical interviews, ensures the developer can explain the solution's algorithmic complexity and edge cases. We demonstrate the feasibility of this approach through the self-governed development of **Brains-Back**, a CLI tool implemented using the proposed pipeline and configures virtually any project to also be governed by it. We position this work at the intersection of Software Engineering and Cognitive Science, arguing that in the age of AI, the primary artifact of value is the developer's sharpened understanding.

Keywords— Large Language Models, Cognitive Debt, Software Engineering, Human-in-the-Loop, Developer Productivity

Putting Brains Back in the Loop:
A Mastery-Aware Pipeline for LLM-Assisted Software Development

Vicente Neto
Pontifícia Universidade Católica do Rio de Janeiro (PUC-Rio)
vneto@tecgraf.puc-rio.br

Alessandro Garcia
Pontifícia Universidade Católica do Rio de Janeiro (PUC-Rio)
afgarcia@inf.puc-rio.br

February 5, 2026

Abstract

The advent of Large Language Models (LLMs) and the tools they enable have dramatically reduced the cost of code production, effectively allowing developers to create and increment software artifacts with unprecedented ease. However, this convenience introduces undesirable side effects, notably the accumulation of *Cognitive Debt*. This debt arises from the removal of friction—a mechanism that, while uncomfortable, is the very fabric of knowledge construction and retention. Developers can now merge functional code without forming a rigorous *Mental Model* of its logic, leading to systems that are fast to grow but increasingly difficult to maintain. To the best of our knowledge, no existing workflow addresses this challenge head-on.

This paper challenges the prevailing trend of "frictionless" development, arguing that developer expertise thrives not on ease, but on the active exercise of solving hard problems. We propose **Mastery-Aware Pipeline**, a workflow that transforms AI-assisted software production into a disciplined cognitive challenge, re-establishing the developer as a software designer rather than a passive reviewer.

The workflow comprises a pipeline supported by AI Agents with two critical stages: (1) **Formal Specification**, achieved through a `TODO.md` artifact that humans—and humans only—must author to guide development; and (2) **Proof-Of-Mastery**, which requires the developer to process the AI-generated code and `REPORT.md` to produce a `REACTO.md` artifact. This structure, inspired by the homonymous technique used in technical interviews, ensures the developer can explain the solution's algorithmic complexity and edge cases. We demonstrate the feasibility of this approach through the self-governed development of **Brains-Back**, a CLI tool implemented using the proposed pipeline and configures virtually any project to also be governed by it. We position this work at the intersection of Software Engineering and Cognitive Science, arguing that in the age of AI, the primary artifact of value is the developer's sharpened understanding.

Keywords— Large Language Models, Cognitive Debt, Software Engineering, Human-in-the-Loop, Developer Productivity

1 Introduction

The software industry is currently optimizing for velocity: tools like GitHub Copilot and other AI assistants powered by Large Language Models (LLMs) integrate deeply into Integrated Development Environments (IDEs) to minimize keystrokes and, while studies confirm massive productivity gains [4], they also warn of a reduction in the variance and depth of thought applied to coding tasks. While comprehensive evidence is still emerging, early examples and anecdotal reports suggest a trend where developers offload cognitive work to the AI. We posit that we are witnessing the emergence of the ***Cognitive Bypass***: a phenomenon where the human acts as a passive reviewer rather than an active creator, approving output in a rubber-stamping fashion without synthesizing the underlying logic (Figure 1).

1

Artigo em construção

Atenção: Cota de uso do copilot

- A cota de uso do copilot free é bastante limitada.
- Caso você tenha utilizado bastante de sua cota, recomenda-se utilizar uma conta diferente do GitHub onde essa cota não tenha sido consumida.
- As contas trial do GitHub foram suspensas temporariamente esta semana, sem previsão de volta devido à uma vulnerabilidade

DICA IMPORTANTE:

- É possível solicitar uma conta "premium" do GitHub destinada à estudantes:

https://github.com/education/students

- É necessário comprovar que você é estudante, então siga as instruções da documentação
- Ter um email @aluno.puc-rio.br ou @puc-rio.br pode facilitar:

https://servicos.rdc.puc-rio.br/

Atenção: Cota de uso do copilot

- **Para obter o comprovante de matrícula no sauonline:**

https://www.puc-rio.br/puconline/

O experimento: TicTacToe

viccsneto/tictactoe_experiment

github.com/viccsneto/tictactoe_experiment

viccsneto / tictactoe_experiment

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security and quality | Insights | Settings

tictactoe_experiment Public

Pin | Watch 0 | Fork 1 | Star 0

master | 1 Branch | 0 Tags | Go to file | Add file | Code

viccsneto Initial commit 24d0cc3 · 6 hours ago 1 Commit

.github	Initial commit	6 hours ago
.vscode	Initial commit	6 hours ago
__tools__/enabled.github	Initial commit	6 hours ago
artifacts	Initial commit	6 hours ago
brainsback	Initial commit	6 hours ago
tests	Initial commit	6 hours ago
.gitignore	Initial commit	6 hours ago
README.md	Initial commit	6 hours ago
game.js	Initial commit	6 hours ago
index.html	Initial commit	6 hours ago
pipeline.bat	Initial commit	6 hours ago
pipeline.sh	Initial commit	6 hours ago
script.js	Initial commit	6 hours ago
style.css	Initial commit	6 hours ago
tests.html	Initial commit	6 hours ago

About

No description, website, or topics provided.

Readme
Activity
0 stars
0 watching
1 fork

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors

No contributors

Languages

JavaScript 68.7% | CSS 13.3% | HTML 10.9% | Shell 3.6% | Batchfile 3.5%

Suggested workflows

https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe - Instruções

README

🕹️ TicTacToe (Jogo da Velha) - Experimento de IA Generativa

Este projeto é parte de uma pesquisa educacional para investigar o **débito cognitivo no desenvolvimento de software utilizando IA Generativa**. A conclusão das atividades propostas neste repositório vale **3 pontos da ATV2**.

📄 Termo de Consentimento

Na pasta raiz deste projeto, dentro de `artifacts/consentimento.md`, você encontra o formulário de consentimento. Antes de iniciar, leia e decida se você concorda ou não com o uso dos seus dados (que serão devidamente anonimizados) no experimento científico para contribuir com a pesquisa.

🛠️ Configuração Inicial (Fork e Upstream)

Para iniciar o trabalho, você deve criar a sua própria versão (fork) do repositório base e configurar o upstream:

1. Acesse o repositório original e clique no botão **Fork**: https://github.com/viccsneto/tictactoe_experiment
2. Clone o repositório "forkado" para o seu ambiente local:

```
git clone https://github.com/SEU_USUARIO/tictactoe_experiment.git
cd tictactoe
```

3. Configure o repositório original como *upstream* para manter seu ambiente sincronizado (e possibilitar a PULL REQUEST):

```
git remote add upstream https://github.com/viccsneto/tictactoe_experiment
```

📝 Tarefas do Experimento

O contexto de desenvolvimento envolverá a resolução de duas tarefas usando a assistência do GitHub Copilot.

Tarefa 1: Emojis no lugar de X e O

https://github.com/viccsneto/tictactoe_experiment/blob/master/README.md

O experimento: TicTacToe - Instruções

Para iniciar o trabalho, você deve criar a sua própria versão (fork) do repositório base e configurar o upstream:

1.Acesse o repositório original e clique no botão **Fork**: https://github.com/viccsneto/tictactoe_experiment

2.Clone o repositório "forkado" para o seu ambiente local e configure o upstream:

```
git clone https://github.com/SEU_USUARIO/tictactoe_experiment.git

cd tictactoe

git remote add upstream https://github.com/viccsneto/tictactoe_experiment
```

https://github.com/viccsneto/tictactoe_experiment

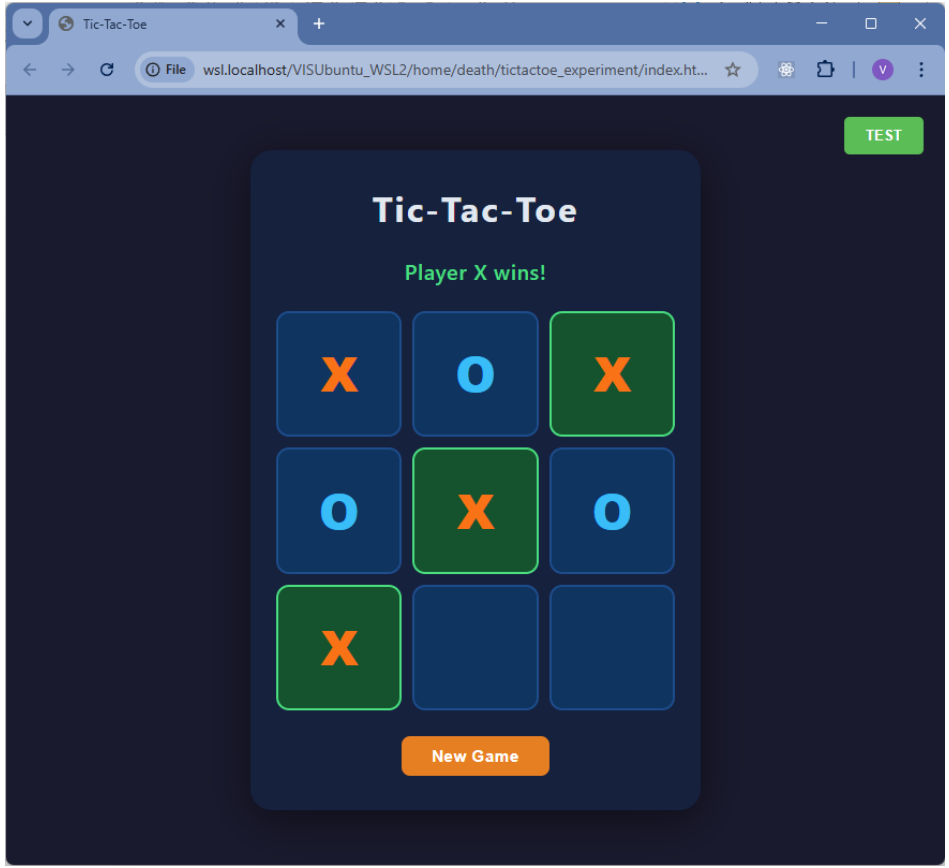
O experimento: TicTacToe - Instruções

Termo de Consentimento

Na pasta raiz deste projeto, dentro de `artifacts/consentimento.md`, você encontra o formulário de consentimento. Antes de iniciar, leia e decida se você concorda ou não com o uso dos seus dados (que serão devidamente anonimizados) no experimento científico para contribuir com a pesquisa.

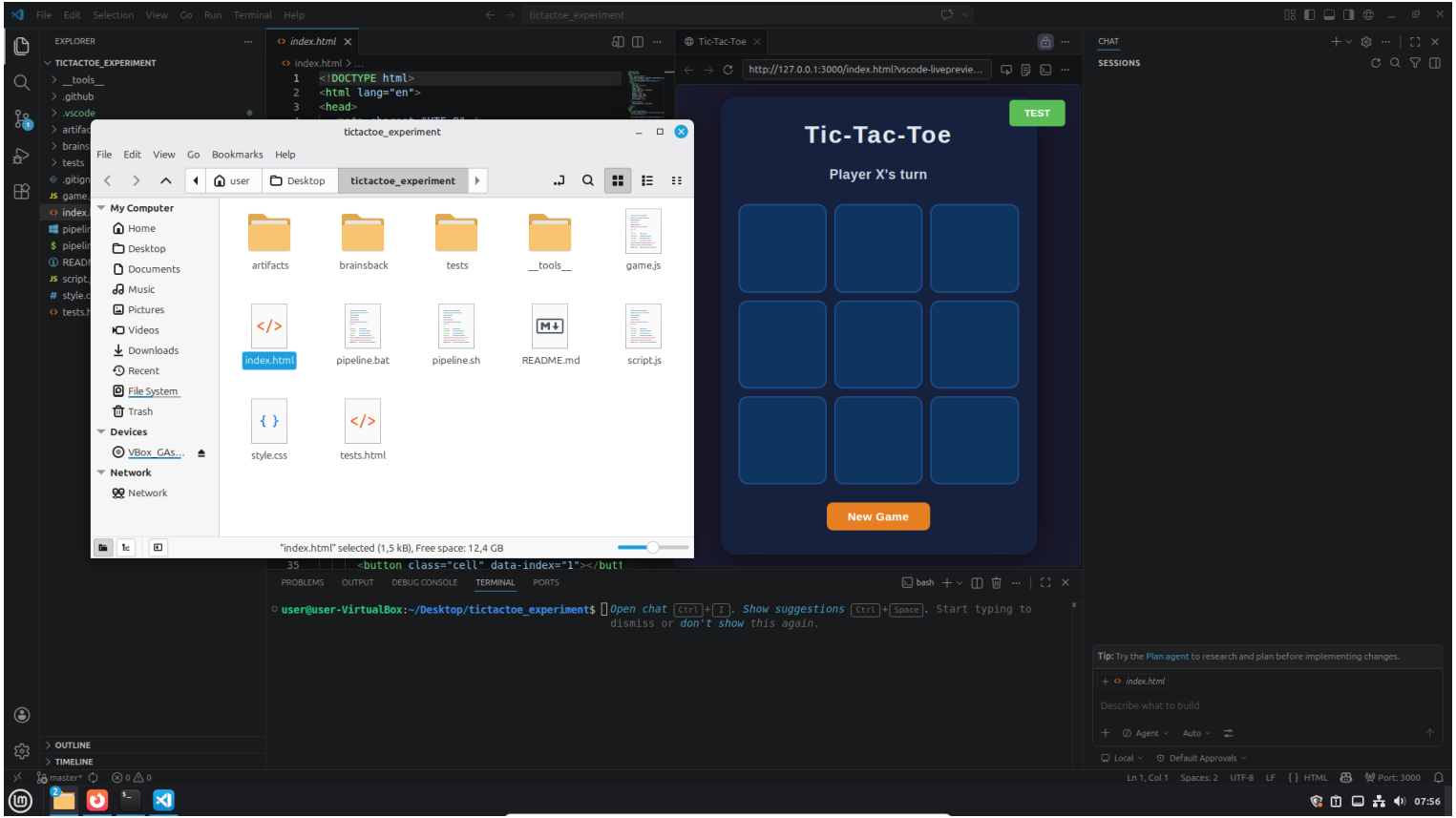
https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe



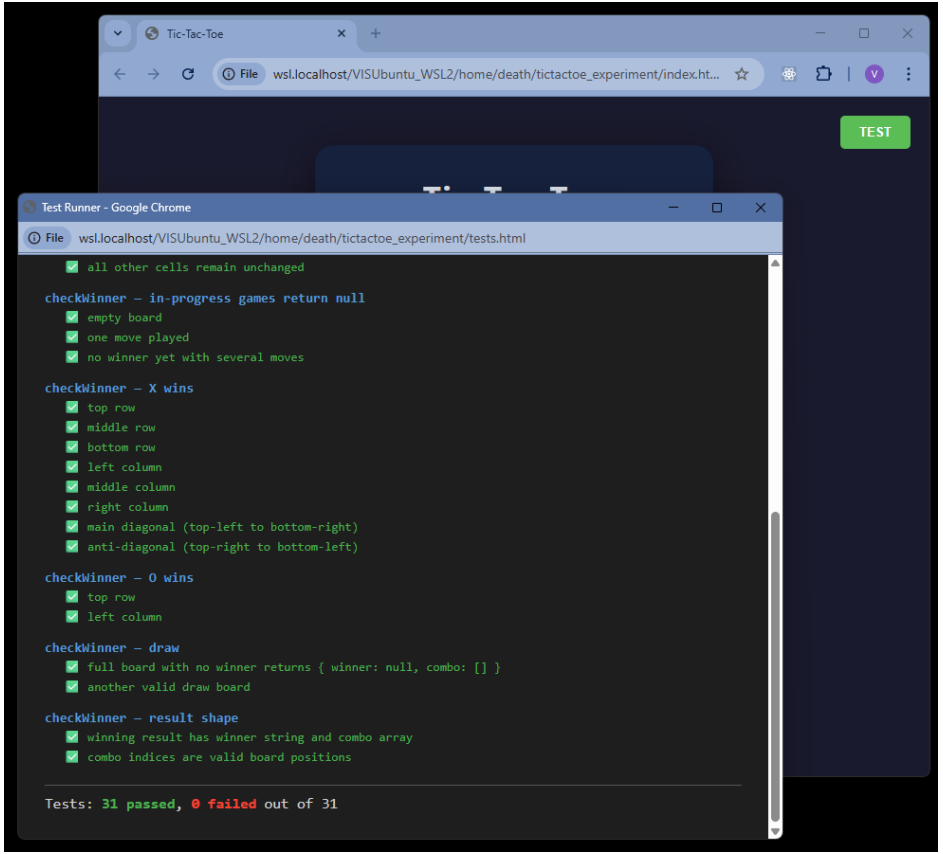
https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe



A aplicação pode ser rodada diretamente do arquivo index.html

O experimento: TicTacToe



Os testes são executados a partir da própria interface

O experimento: TicTacToe - Instruções

Tarefa 1: Emojis no lugar de X e O

Nesta primeira missão, você deve **substituir os símbolos tradicionais** `X` **e** `O` do jogo da velha por emojis:

- Onde era `X`, passará a ser 😸 (cat face).
- Onde era `O`, passará a ser 🐶 (dog face).

⚠️ **Regras da Tarefa 1 (Pipeline MasteryAware):**

- Durante esta etapa, o assistente Copilot operará sob as regras do pipeline **MasteryAware**, projetado para garantir que você se mantenha como o "designer do software" em vez de apenas um revisor passivo de código gerado.
- **Passo 1: Todo.md (Especificação Formal):** Antes do Copilot gerar qualquer código para você, **VOCÊ** deve preencher manualmente o arquivo `brainsback/TODO.md`. Ele serve como sua lousa para articular o problema, os requisitos e os critérios de sucesso. O Copilot é proibido de editar este arquivo; se você tentar pedir ajuda sem preenchê-lo, ele se recusará a codificar.
- **Passo 2: Implementação Assitida:** Uma vez que o `TODO.md` for preenchido com suas diretrizes, você e o Copilot formarão uma dupla (*Pair Programming*). Interaja naturalmente com o agente para atingir o objetivo, realizando testes contínuos da solução. O Copilot eventualmente gerará um artefato autônomo de relatório (`brainsback/REPORT.md`) resumindo o que construiu.

https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe - Instruções

Tarefa 1: Emojis no lugar de X e O

- **Passo 3: REACTO.md (Prova de Domínio):** Após a implementação concluir, **VOCÊ** elaborará ativamente o artefato `brainsback/REACTO.md` unindo a análise do sumário gerado pela IA com a conferência das linhas de código. O formato `REACTO` significa:
  - **R (Repeat):** Reafirme o problema com suas próprias palavras.
  - **E (Examples):** Dê exemplos de entrada/saída (ou ação/resultado) em casos de teste na vida real.
  - **A (Approach):** Qual estratégia em alto nível foi utilizada para solucionar o problema?
  - **C (Code):** Exponha os trechos mais cruciais, que funções realizam o quê, em quais arquivos estão definidas e quem as chama.
  - **T (Test):** Rastreie os resultados relatando testes e cenários experimentados (manuais, automáticos).
  - **O (Optimize):** Sugira melhorias de complexidade ou discuta os "*trade-offs*" estabelecidos na implementação. Nem sempre isso se aplica.
- Você pode e deve fazer qualquer questionamento ou tirar dúvidas com o próprio GitHub Copilot e/ou com o professor durante o processo.
- **Antes de fazer o commit**, você **deve perguntar ao Copilot**: "*Minha tarefa está pronta para commit e de acordo com as regras do pipeline mastery-aware?*"
- Apenas quando o Copilot validar sua conclusão (verificando os artefatos) e der o "OK", você deverá realizar o commit de todos os arquivos modificados nesta primeira etapa.

https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe - Instruções

Tarefa 2: Implementação do Placar (Score)

Nesta segunda missão, você deverá **implementar um sistema de pontuação**, contabilizando e exibindo as vitórias do Gato (🐱) e do Cachorro (🐶).

⚠️ **Regras da Tarefa 2:**

- **Importante:** Antes de escrever o código ou conversar com o Copilot sobre essa tarefa, **desabilite o pipeline**. Para isso, execute no terminal o script adequado ao seu Sistema Operacional:

Linux e Mac

```
./pipeline.sh OFF
```

Windows

```
pipeline.bat OFF
```

- A partir de agora, o desenvolvimento não será mais monitorado pelo pipeline MasteryAware. Você deve desenvolver toda a solução interagindo livremente com o Copilot pelo chat.
- Garanta que a sua implementação esteja funcionando corretamente através de testes manuais e valide se os testes unitários do projeto continuam passando.
- Assim que o placar estiver totalmente funcional, realize um novo commit contendo as modificações referentes à segunda tarefa.

https://github.com/viccsneto/tictactoe_experiment

O experimento: TicTacToe - Instruções

🔍 Revisão final e Pull Request

Após as implementações, finalizaremos com uma dinâmica de perguntas.

- Solicite explicitamente ao Copilot no chat: `Quero iniciar a revisão socrática`
- O agente revisor irá analisar os códigos que você produziu e realizará algumas perguntas para compreender sua linha de raciocínio e testar seu domínio sobre o código.
- Essa etapa irá gerar mais dois artefatos (arquivos) consolidados na pasta do projeto ao fim da interação.
- **Atenção:** Você apenas poderá commitar seus arquivos finais e submeter o **Pull Request** para aprovação do repositório original da disciplina quando o revisor socrático concluir a revisão e afirmar explicitamente que *está tudo pronto para o PR*.

https://github.com/viccsneto/tictactoe_experiment

Dúvidas?

https://github.com/viccsneto/tictactoe_experiment

Assistentes de AI generativa e o risco de Débito Cognitivo - Exercício

INF1028 e INF1045

Alessandro Garcia
Vicente Neto